05 EXPRESS vs NESTJS — WHERE BUSINESS LOGIC LIVES

Short answer: in plain Express, business logic is commonly written directly inside the controller (route handler) — because Express gives you nothing that forces otherwise. NestJS architecturally forces logic OUT of the controller and into a service.

WHY THE DIFFERENCE EXISTS

Express is a minimal, unopinionated framework. It only gives you:

- app.get() / app.post() / etc. (routing)
- req / res objects
- middleware chain

There is no built-in concept of a "service", no dependency injection container, nothing that separates HTTP handling from business logic. Whatever structure exists is 100% up to the developer's discipline.

NestJS is opinionated and structural. It gives you:

- @Controller() for routing/HTTP only
- @Injectable() + DI container for business logic (services)
- Modules that wire controllers and services together

Because Nest's architecture is built around Dependency Injection, the natural (and enforced-by-convention) place for logic is the service, not the controller.

EXPRESS EXAMPLE — LOGIC LIVES IN THE CONTROLLER (TYPICAL)

```
// controllers/catsController.js
exports.createCat = async (req, res) => {
  const { name, breed, age } = req.body;

  // business rule — validation
  if (age < 0) {
    return res.status(400).json({ error: 'invalid age' });
  }

  // business rule — DB access
  const cat = await Cat.create({ name, breed, age });

  // HTTP response
  res.status(201).json(cat);
};

// routes/catsRoutes.js
router.post('/cats', catsController.createCat);
```

All three concerns — validation, DB access, response shaping — sit in one function. Nothing stops this; it's just how most Express apps are written unless the team deliberately adds a service layer.

EXPRESS — HOW YOU *COULD* ADD A SERVICE LAYER MANUALLY (OPT-IN)

```
// services/catsService.js
exports.createCat = async ({ name, breed, age }) => {
  if (age < 0) throw new Error('invalid age');
  return Cat.create({ name, breed, age });
};

// controllers/catsController.js
const catsService = require('../services/catsService');

exports.createCat = async (req, res) => {
  try {
    const cat = await catsService.createCat(req.body);
    res.status(201).json(cat);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
};
```

This is possible and considered good practice in larger Express codebases — but it's entirely optional. Express will never complain if you skip it.

NESTJS EXAMPLE — LOGIC IS FORCED OUT OF THE CONTROLLER

```
// cats.service.ts
@Injectable()
export class CatsService {
  create(dto: CreateCatDto) {
    if (dto.age < 0) throw new BadRequestException('invalid age');
    return this.catsRepo.save(dto);
  }
}

// cats.controller.ts
@Controller('cats')
export class CatsController {
  constructor(private readonly catsService: CatsService) {}

  @Post()
  create(@Body() dto: CreateCatDto) {
    return this.catsService.create(dto);   // no logic here at all
  }
}
```

The controller CANNOT easily reach a database or run business rules inline in idiomatic Nest — the whole framework (modules, DI, decorators) nudges every developer toward the same structure, so large teams end up consistent without needing a style guide to enforce it.

SIDE-BY-SIDE SUMMARY

```
ASPECT                         | EXPRESS                          | NESTJS
-------------------------------|----------------------------------|-------------------------------
```

```
Service layer                  | Optional, manual                | Built-in (@Injectable + DI)
Where logic usually ends up    | Directly in the route handler   | In a separate service
Enforced by the framework?     | No — pure developer discipline  | Yes — architecture requires it
Dependency Injection           | Not built in (roll your own)    | First-class, automatic
Consistency across a big team  | Depends entirely on code review | Enforced by structure itself
Is "logic in controller" wrong? | Not wrong, just common         | Goes against the framework's design
```

★ RULE OF THUMB

Coming from Express, the adjustment when learning Nest is: "Just because I *can* fit logic in the controller doesn't mean I should in Nest — the framework already gives me a service for that, so use it."

See also: theory/02-controller.txt, theory/03-service.txt, theory/04-controller-vs-service.txt